PROJECT AND TEAM INFORMATION

Project Title

Smart Hostel Management & Complaint Resolution System

Student / Team Information

Team Name: *Team ID:*	*Hostel- Nexus* *DSCPP-III-2026-T269*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	Raj, Harshit - 2510011339 *rajharshit1807@gmail.com* 
Team member 2 *(Last Name, name: student ID: email, picture):*	Negi, Diviya - 2510010573 *diviyanegi51@gmail.com* 
Team member 3 *(Last Name, name: student ID: email, picture):*	Verma, Aditya – 2510380258 av0844769@gmail.com 

PROPOSAL DESCRIPTION

Motivation

Anyone who has lived in our college hostels knows how chaotic the front desk gets during semester check-ins. The warden sits with thick paper registers, manually scribbling room allotments and bed numbers as hundreds of students queue up. It does not take long for mistakes to happen. One register shows bed B in Room 204 as occupied, another notebook shows it empty, and sometimes vacant rooms go unnoticed for an entire month while students hunt for accommodation. Maintenance is even worse. When a tap breaks, a switchboard sparks, or the water cooler breaks down, students just write a quick note in a dog-eared diary at the gate or tell the caretaker in passing. There is no tracking. A dangerous electrical short circuit and a loose cupboard handle get dumped into the exact same page, handled purely on a first-come basis simply because the carpenter happened to read the notebook before the electrician.

Student outpasses, weekend leaves, and visitor sign-ins create another pile of loose paperwork. If a parent calls or the warden needs to confirm whether a student is present in their room or went home, someone has to physically flip through pages of handwritten entries. As the hostel expands to several hundred students, this paper-based approach completely breaks down. Trying to track a student's past room history, check previous maintenance records, or audit who visited whom becomes a massive headache because information sits scattered across disconnected books.

Our goal in this project is to eliminate this manual paperwork by building a single, reliable console system. It brings student records, bed vacancies, repair requests, and visitor entries into one organized application. Critical maintenance issues jump straight to the top of the queue so caretakers fix hazards immediately, while standard search queries take less than a second. Solving this is not just about reducing clerical work—it directly improves daily student safety and hostel living conditions.

State of the Art / Current solution

Today, hostel administration in our college still relies on a messy combination of paper registers, loose Excel sheets on the office computer, and unstructured WhatsApp groups. When students arrive, room allotments are written down in paper books. Vacancy details get updated only when the clerk remembers to check, which often causes duplicate bed allotments or leaves empty rooms unoccupied for weeks. For repairs, complaints get written sequentially in a reception register. Emergency electrical or plumbing faults wait in line right behind minor carpentry fixes, with zero automated priority. Even where staff use spreadsheets, the files remain completely disconnected—student lists, gate passes, and repair logs never sync. To track down a resident's history or review past complaints, the warden has to manually dig through old notebooks. Heavy ERP software exists in corporate universities, but it is far too expensive, slow, and complicated for individual hostel blocks. Our team is building a practical, lightweight terminal tool in C and C++ that automates room tracking, puts urgent repairs first, and keeps all student and hostel records connected without bloated software.

Project Goals and Milestones

General Goals:

- Centralized CLI hub: Replace separate paper registers (room lists, gate passes, maintenance) with one integrated console application.
- Double-booking prevention: Immediate memory checks whenever a bed is assigned so two students never get allotted the same bed.
- Emergency-first triage: Automatically push dangerous hazards (short circuits, water leaks) to the front of the repair queue.
- Sub-second lookups: Find any resident or room record in O(1) average time without scanning through files sequentially.
- Gatekeeper module: Log student weekend outpasses and parent/visitor check-ins directly against the student's roll number.
- Tenancy audit trail: Maintain dynamic linked lists of past room occupants and resolved maintenance issues for semester review.
- Pure C and C++ OOP: Write core DSA structures in ANSI C, and wrap application logic in clean, object-oriented C++ classes.

Milestones:

- Milestone 1: Requirement gathering and defining data schemas for students, rooms, complaints, and visitor logs.
- Milestone 2: Construct core C structs and establish the C++ class hierarchy using an abstract Person base class.
- Milestone 3: Implement student enrollment and bed allocation modules utilizing custom hash tables for O(1) lookups.
- Milestone 4: Build dual-queue ticket processing: priority queue for emergencies and a standard FIFO queue for routine repairs.
- Milestone 5: Create visitor entry logs and student leave/outpass tracking linked directly to resident profiles.
- Milestone 6: Add sorting routines (QuickSort/MergeSort) and build singly linked lists for room occupancy history.
- Milestone 7: Write file handling routines using CSV text storage to save and reload records across program runs.
- Milestone 8: Terminal UI integration, menu navigation, and edge case testing (full capacity, invalid room IDs, duplicate records).

Project Approach (3 pts)

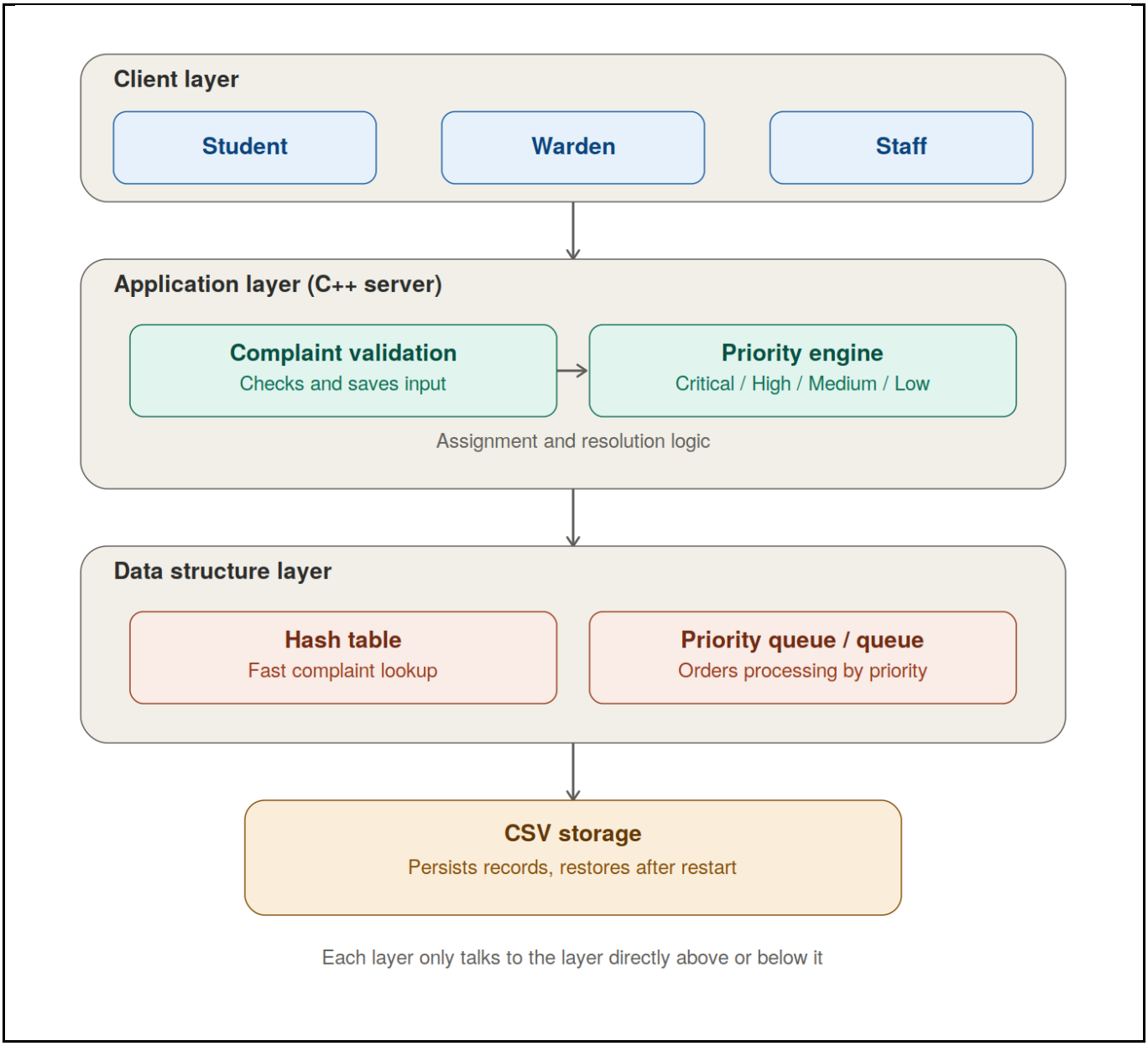
We divided our implementation into two clean layers. To satisfy our DSA course requirements, we write all the fundamental data structures from scratch in ANSI C using custom structs and memory pointers. Around these C

components, we use C++ to manage console menus, handle program flow, and model real-world hostel entities using object-oriented design.

Searching through hundreds of room records in a basic array is slow, so we index students and rooms using a custom hash table keyed on roll numbers and room numbers. This lets the warden retrieve any student profile or check bed occupancy in instant O(1) average time. For repairs, we implemented two separate queues. If someone reports a burst water pipe or a sparking electrical line, that ticket goes into a binary heap priority queue based on severity, so emergency repairs always stay at the front. Regular requests—like fixing a table drawer or requesting room whitewash—sit in a standard first-in, first-out queue. Because student batches change every year, each room keeps a singly linked list that logs who stayed in that room across past terms. We also wrote QuickSort and MergeSort routines to sort complaints by date or priority tag, with binary search used whenever we need to query sorted lists.

On the C++ side, our code represents hostel entities using five main classes: Student, Room, Complaint, Visitor, and Warden. Both Student and Warden inherit common details—like full name, mobile number, and ID—from an abstract Person base class. We defined a virtual displayDetails() method in Person and overrode it in child classes to show runtime polymorphism in action. All member variables stay private, modified only through getters and setters to practice strict data encapsulation. Parameterized constructors handle object setup cleanly, and try-catch blocks guard against user errors like entering duplicate roll numbers or allocating a non-existent room. To keep records between program runs, our file-handling functions read and write comma-separated CSV files via fstream. We deliberately avoided using STL vectors or map containers for the core logic, proving our hand-built C data structures do all the heavy lifting.

System Architecture (High Level Diagram)



Project Outcome / Deliverables

The final outcome of this project will be a working C++ console application for the Smart Hostel Management System, delivering:

- Student onboarding and profile lookups by roll number or name in sub-second time.
- Live room allotment grid with vacancy tracking to actively stop accidental double-bookings.
- Priority repair manager routing emergency safety repairs through a priority queue and routine fixes via a

normal FIFO queue.

- Gatekeeper module for logging visitor entries and student leave/outpass movements linked to resident IDs.
- Multi-criteria search and sorting tools to organize records by urgency, room number, or submission date.
- CSV text file storage to preserve all student, room, and complaint data across program restarts.
- Audit history tracking using linked lists to review past room occupancies and resolved complaint logs.
- Interactive CLI demonstration covering boundary conditions, such as full hostel capacity, bad room inputs, and duplicate student IDs.

By running this terminal tool, our faculty evaluator can directly see how low-level C pointers, structs, and algorithms interact with C++ OOP classes to run a complete hostel management workflow.

Assumptions

For this semester project, we are limiting the scope to a single hostel building (around 50 rooms and 150 beds). Since we do not have access to our university's internal ERP or live database servers, all student profiles, roll numbers, and room entries used in our demo are test records we created ourselves. There are no biometric thumb scanners, RFID turnstiles, or online fee portals connected to this code. Complaint priority (high, medium, low) is simply picked by the student or staff from a console menu rather than calculated by sensors. Everything runs offline in the terminal to keep our code focused purely on C data structures and C++ classes.

References

- cppreference.com - C++ language and standard library reference (classes, inheritance, polymorphism, exception handling, STL): https://en.cppreference.com/
- GeeksforGeeks - Data Structures and Algorithms: Hash Table, Priority Queue, Queue, Linked List, Searching and Sorting algorithms: https://www.geeksforgeeks.org/data-structures/
- GeeksforGeeks - File Handling in C/C++: https://www.geeksforgeeks.org/file-handling-c-classes/
- Design and implementation of automated secure web-based hostel management system with automated room allocation and complaint tracking: https://www.researchgate.net/publication/401357924_Design_and_implementation_of_automated_secure_web-based_hostel_management_system_with_automated_room_allocation_and_complaint_tracking
- Development of Web-Based Hostel Management System: https://www.researchgate.net/publication/389336918_Development_of_Web-Based_Hostel_Management_System
- Hybrid Technology Based Smart Hostel Management System Using Artificial Intelligence and Internet of Things: https://www.researchgate.net/publication/369283563_Hybrid_Technology_Based_Smart_Hostel_Management_System_Using_Artificial_Intelligence_and_Internet_of_Things